

Carrera:
**Licenciatura en Ingenieria en Programacion y
Web Master**

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
PROYECTO DE PLATAFORMA VIRTUAL

Alumno(s):
Gerardo Alexander Diaz Leon

Fecha:
24/05/2026

Introducción

En la actualidad, los sistemas de streaming musical se han convertido en una de las principales formas de consumo de contenido digital alrededor del mundo. Plataformas similares a Spotify permiten a millones de usuarios acceder fácilmente a música, podcasts y contenido multimedia desde distintos dispositivos conectados a internet. Debido a su complejidad tecnológica y gran cantidad de usuarios simultáneos, estos sistemas requieren un proceso de desarrollo bien estructurado para garantizar calidad, seguridad y eficiencia.

El Ciclo de Vida del Desarrollo de Software (SDLC) es una metodología que permite organizar y controlar cada etapa involucrada en la creación de un sistema informático, desde la planeación inicial hasta el mantenimiento posterior a su implementación. Gracias a este proceso, es posible identificar requerimientos, diseñar arquitecturas adecuadas, desarrollar funcionalidades, realizar pruebas y mantener el sistema actualizado de forma continua.

En este trabajo se analizará el desarrollo de un sistema ficticio de streaming musical llamado SoundFlow, inspirado en plataformas modernas de música en línea. Se estudiará cómo se aplicarían las distintas etapas del SDLC, incluyendo planeación, análisis de requerimientos, diseño, desarrollo, pruebas, implementación y mantenimiento. Además, se reflexionará sobre la importancia de cada etapa y el impacto que tiene el SDLC en la calidad y organización de los proyectos de software modernos.

Descripción del Sistema

1. Nombre del sistema: SoundFlow.
2. ¿Qué problema resuelve? SoundFlow resuelve la dificultad de acceder fácilmente a música y contenido de audio desde diferentes dispositivos. Muchas personas buscan una plataforma rápida, organizada y personalizada para escuchar sus canciones favoritas sin necesidad de descargar archivos manualmente o almacenar gran cantidad de contenido en sus equipos.
3. Tipo de usuarios.

Usuarios gratuitos.

1. Escuchan música con anuncios publicitarios.
2. Tienen acceso limitado a ciertas funciones.
3. Pueden crear playlists básicas y explorar contenido.

Usuarios Premium

1. Pagan una suscripción mensual.
2. Escuchan música sin anuncios.
3. Descargan canciones para escuchar sin conexión.
4. Obtienen mejor calidad de audio y funciones avanzadas.

Artistas y creadores

1. Publican canciones, álbumes y podcasts.
2. Administran su contenido y estadísticas.

3. Interactúan con sus seguidores dentro de la plataforma.

Administradores del sistema

1. Supervisan el funcionamiento de la plataforma.
2. Gestionan usuarios, contenido y reportes.
3. Mantienen la seguridad y estabilidad del sistema.

4. Funcionalidades Principales

Reproducción de música en línea

Permite escuchar canciones, álbumes y podcasts mediante streaming desde cualquier dispositivo conectado a internet.

Sistema de búsqueda avanzada

Los usuarios pueden buscar artistas, canciones, géneros musicales o playlists de manera rápida.

Creación y gestión de playlists

Los usuarios pueden crear, editar y compartir listas de reproducción personalizadas.

Recomendaciones inteligentes

El sistema analiza los gustos musicales del usuario para sugerir nuevo contenido mediante algoritmos de recomendación.

Descarga de contenido

Los usuarios Premium pueden descargar canciones y escucharlas sin conexión a internet.

Sincronización multidispositivo

La cuenta mantiene preferencias, historial y playlists sincronizadas entre teléfono, computadora y otros dispositivos.

Interacción social

Los usuarios pueden seguir artistas, compartir música y ver tendencias populares dentro de la plataforma.

Panel para artistas

Los creadores pueden subir contenido, visualizar estadísticas de reproducciones y administrar su perfil.

5. Importancia del sistema

SoundFlow es importante porque ofrece una forma moderna, rápida y accesible de consumir música digital. El sistema facilita el acceso legal a millones de canciones y mejora la experiencia del usuario mediante recomendaciones personalizadas y sincronización en múltiples dispositivos.

Además, ayuda a artistas independientes a difundir su contenido y llegar a nuevas audiencias sin depender exclusivamente de medios tradicionales.

Desde el punto de vista tecnológico, representa un sistema de gran escala basado en servicios en línea, almacenamiento en la nube y manejo simultáneo de miles de usuarios.

Etapas de Planeación

La etapa de planeación es una de las fases más importantes dentro del desarrollo del sistema, ya que permite definir los objetivos, recursos, tiempos y estrategias necesarias para crear la plataforma de streaming musical SoundFlow.

El desarrollo de este sistema sería necesario debido al crecimiento del consumo de contenido digital y la necesidad de ofrecer una plataforma moderna, accesible y personalizada para escuchar música y podcasts en línea.

1. Objetivos del sistema

Objetivo general

Desarrollar una plataforma de streaming musical que permita a los usuarios escuchar música y podcasts desde distintos dispositivos de forma rápida, segura y personalizada.

Objetivos específicos

1. Permitir la reproducción de música en línea.
2. Implementar cuentas gratuitas y Premium.
3. Crear un sistema de recomendaciones musicales.
4. Permitir la creación y gestión de playlists.
5. Ofrecer sincronización entre dispositivos.
6. Proporcionar un panel de administración para artistas y administradores.

Alcance general

El sistema estará disponible inicialmente como: Aplicación web y Aplicación móvil (Android y iOS).

El proyecto tendrá:

1. Registro e inicio de sesión de usuarios.
2. Reproducción de contenido multimedia.
3. Motor de búsqueda musical.
4. Gestión de playlist.
5. Sistema de suscripciones.
6. Base de datos de usuarios y canciones.
7. Panel administrativo.

Recursos necesarios

En recursos tecnológicos para desarrollo del sistema se requerirá lo siguiente:

1. Computadoras de desarrollo.
2. Servidores en la nube.
3. Base de datos.
4. Frameworks de desarrollo web y móvil.
5. Herramientas de diseño y control de versiones.
6. Servicios de almacenamiento multimedia.
7. Conexión estable a internet.

Riesgos principales.

Durante el desarrollo del proyecto podrían surgir distintos problemas como fallas de seguridad, sobrecarga del sistema, retrasos en el desarrollo, problemas de compatibilidad, errores en el sistema (bugs, optimización, etc.) y problemas legales relacionados con derechos de autor.

Tiempo estimado.

El desarrollo inicial del sistema podría tomar aproximadamente entre 9 y 13 meses, dependiendo del tamaño del equipo y la complejidad del sistema. A continuación una pequeña distribución aproximada:

1. Planeación y análisis: 1 mes.
2. Diseño del sistema: 1 mes.
3. Desarrollo: 5 a 7 meses.
4. Pruebas y correcciones: 1 a 2 meses.
5. Implementación final: 1 mes.

Costos aproximados.

El desarrollo del sistema requiere una inversión considerable debido al uso de servidores, desarrollo de software, almacenamiento multimedia y mantenimiento de la plataforma. A continuación un desglose de los precios:

1. Planeación y análisis del proyecto: \$40,000 MXN.
2. Diseño de interfaz UI/UX: \$35,000 MXN.
3. Desarrollo frontend (web y móvil): \$120,000 MXN.
4. Desarrollo backend: \$100,000 MXN.
5. Base de datos y configuración de servidores: \$45,000 MXN.
6. Sistema de autenticación y seguridad: \$30,000 MXN.
7. Implementación de reproducción multimedia \$50,000 MXN.
8. Almacenamiento en la nube: \$25,000 MXN.
9. Pruebas y control de calidad: \$35,000 MXN.
10. Licencias y herramientas de desarrollo: \$20,000 MXN.
11. Dominio y hosting inicial: \$10,000 MXN
12. Mantenimiento inicial del sistema: \$40,000 MXN.

Total aproximado: \$550,000 MXN

Análisis de Requerimientos.

1. Requerimientos Funcionales

RF-01: **El sistema deberá permitir el registro e inicio de sesión de usuarios.**

RF-02: **El sistema deberá permitir la reproducción de música y podcasts en línea.**

RF-03: **El sistema debe permitir crear editar, y eliminar playlists.**

RF-04: **El sistema debe contar con un buscador para encontrar canciones, artistas, álbumes y podcasts.**

RF-05: **El sistema debe permitir a los usuarios que paguen la suscripción, descargar contenido para escucharlo sin conexión.**

2. Requerimientos no Funcionales.

RNF-01: **El sistema debe proteger la información personal mediante cifrado de datos y contraseñas.**

RNF-02: **La plataforma debe estar disponible las 24 horas del día.**

RNF-03: **El sistema debe ser compatible con navegadores modernos y dispositivos móviles.**

RNF-04: **El tiempo de carga de las canciones no debe superar los 3 segundos con conexión estable.**

RNF-05: **La interfaz debe ser intuitiva, responsiva y fácil de usar.**

Diseño del sistema

1. Arquitectura general.

El sistema podría utilizar una arquitectura cliente-servidor basada en servicios web y almacenamiento en la nube.

Componentes principales:

El Cliente (Frontend) envía solicitudes al servidor y muestra la información al usuario como la aplicación web, aplicación móvil y reproductor multimedia.

El servidor (Backend) se encarga de procesar solicitudes, gestionar usuarios, administrar playlists, controlar autenticación, manejar recomendaciones musicales y conectar con la base de datos.

La base de datos almacena la información de usuarios, canciones, podcasts, historial de reproducción, suscripciones y playlists, a continuación una pequeña tabla de posibles tablas principales:

Tabla	Función
Usuarios	Guarda datos de cuentas y perfiles
Canciones	Almacena información musical
Artistas	Información de creadores de contenido
Playlists	Listas de reproducción creadas
Historial	Registra reproducciones recientes
Suscripciones	Controla cuentas Premium
Podcasts	Información de programas y episodios

El servidor multimedia se utilizará para almacenar archivos de audio, transmitir contenido de Podcasts y optimizar la reproducción sin interrupciones.

El gestor de base de datos serían MySQL o PostgreSQL.

Las tecnologías posibles en Frontend serían las siguientes:

1. HTML5
2. CSS
3. JavaScript
4. React
5. Flutter

Las tecnologías posibles para la lógica del sistema (Backend) serían las siguientes:

1. [Node.js](#)
2. [Express.js](#)

3. PHP
4. Python

La infraestructura y nube, posiblemente se utilicen Amazon Web Services, Google Cloud o Microsoft Azure.

2. Interfaces

La interfaz del usuario permitirá buscar música, reproducir canciones, crear playlists, administrar cuentas y descargar contenido.

La interfaz para artistas permitirá subir canciones y podcasts, ver estadísticas y gestionar perfiles.

La interfaz administrativa permitirá gestionar usuarios, supervisar contenido, revisar reportes y controlar suscripciones y seguridad.

3. Seguridad

Medidas de seguridad posibles:

1. Encriptación de contraseñas.
2. Autenticación segura mediante tokens.
3. Protección contra ataques cibernéticos.
4. RespalDOS automáticos de base de datos.
5. Control de acceso según roles.
6. Monitoreo constante de actividad sospechosa.
7. Uso de conexiones seguras HTTPS.

4. Apis o Servicios externos

Las posibles integraciones están plasmadas en la siguiente tabla:

Servicio	Función
API de pagos	Gestionar suscripciones Premium
API de autenticación	Inicio de sesión con Google o Facebook
API de almacenamiento	Guardar archivos multimedia
API de recomendaciones	Mejorar sugerencias musicales
CDN (Content Delivery Network)	Mejorar velocidad de reproducción
Servicios de analítica	Monitorear actividad y estadísticas

Desarrollo

La etapa de desarrollo consiste en la construcción del sistema mediante programación, integración de tecnologías y pruebas constantes para garantizar el correcto funcionamiento de la plataforma.

1. Roles del equipo

Para el desarrollo del sistema sería necesario un equipo multidisciplinario con diferentes responsabilidades que se plasma en la siguiente tabla:

Rol	Función
Jefe de proyecto	Coordina el proyecto, tiempos y actividades del equipo.
Analista de sistemas	Define requerimientos y estructura del sistema.
Diseñador UI/UX	Diseña interfaces visuales y experiencia del usuario.
Desarrollador frontend	Construye la parte visual de la aplicación.
Desarrollador backend	Programa la lógica del sistema y APIs.
Administrador de base de datos	Diseña y mantiene la base de datos.
Especialista en ciberseguridad	Implementa medidas de seguridad y protección de datos.
Tester o QA	Realiza pruebas y detecta errores del sistema.
DevOps	Gestiona servidores, despliegues y servicios en la nube.

2. Tecnologías de desarrollo

El sistema podría desarrollarse utilizando tecnologías modernas para aplicaciones web y móviles.

Las tecnologías posibles en Frontend serían las siguientes:

1. HTML5
2. CSS
3. JavaScript
4. React
5. Flutter

Las tecnologías posibles para la lógica del sistema (Backend) serían las siguientes:

1. [Node.js](#)
2. [Express.js](#)
3. PHP
4. Python

El gestor de base de datos serían MySQL o PostgreSQL.

Las herramientas adicionales serían Git y GitHub.

3. Organización del trabajo

El trabajo podría organizarse mediante metodologías ágiles como Scrum, permitiendo dividir el proyecto en pequeñas etapas llamadas sprints.

Posible organización

Sprint 1

1. Diseño inicial.
2. Configuración del proyecto.
3. Creación de base de datos.

Sprint 2

1. Sistema de usuarios.
2. Inicio de sesión y autenticación.

Sprint 3

1. Reproducción multimedia.
2. Gestión de playlists.

Sprint 4

1. Sistema Premium.
2. Descargas y sincronización.

Sprint 5

1. Panel administrativo.
2. Recomendaciones musicales

Sprint 6

1. Pruebas finales
2. Corrección de errores.
3. Implementación.

4. Control de Versiones

El control de versiones permitiría registrar cambios realizados por los desarrolladores y trabajar colaborativamente sin perder información y las posibles herramientas serían Git y Github.

5. Posibles dificultades

Dificultades técnicas serían los problemas de rendimiento en streaming, sobrecarga del servidor, errores de sincronización.

Dificultades técnicas.

1. Problemas de rendimientos.
2. Sobrecarga de servidor.
3. Errores de sincronización.
4. Fallas en el sistema de recomendaciones.
5. Compatibilidad con las plataformas.

Dificultades organizacionales.

1. Retrasos de tiempo de entrega.
2. Problemas de comunicación de equipo.
3. Cambios constantes de requerimientos.
4. Incremento de costos.
5. Errores detectados en etapas avanzadas.

Dificultades de seguridad.

1. Riesgo de robo de datos.
2. Ataques cibernéticos.

3. Problemas de autenticación.
4. Vulnerabilidad de APIs.

Pruebas

Antes de liberar el sistema, es necesario hacer diferentes pruebas para garantizar que la plataforma funcione correctamente, sea segura y ofrezca una buena experiencia para el usuario.

1. Pruebas funcionales: Para las pruebas funcionales verificaremos si podemos crear una cuenta correctamente, iniciar sesión, comprobar si las canciones se reproducen sin errores o retrasos, verificar la creación de playlists, descargas para usuarios con suscripción y el panel administrador. Todo esto para verificar que el sistema corra correctamente según los requerimientos establecidos
2. Pruebas de seguridad: Para las pruebas de seguridad verificaremos el acceso seguro del sistema, validar protección de datos personales, evitar manipulación de base de datos y proteger cuentas del usuario. Todo esto para evitar comprometer los datos personales de los usuarios.
3. Pruebas de rendimiento: Para las pruebas de rendimiento simularemos miles de usuarios conectados simultáneamente, medir tiempo de carga de canciones, analizar uso de memoria y procesador, verificar funcionamiento continuo durante horas largas y evaluar comportamiento bajo sobrecarga extrema.
4. Pruebas de integración: Para las pruebas de integración verificaremos comunicación entre interfaz y servidor, confirmar almacenamiento y recuperar información, validar funcionamiento de pagos y autenticación externa, revisar transmisión correcta del contenido multimedia y confirmar sincronización de pagos y beneficios de usuario.
5. Pruebas de usuarios: Para las pruebas de usuarios verificaremos que la interfaz sea intuitiva, evaluar comodidad al escuchar música, revisar funcionamiento en teléfonos y tabletas, confirmar adaptación a diferentes pantallas y obtener retroalimentación antes del lanzamiento.

Implementación

1. La infraestructura

La infraestructura del sistema debe ser capaz de soportar miles de usuarios reproduciendo contenido simultáneamente, los componentes serán los siguientes:

Componente	Función
Servidores de aplicaciones	Procesan solicitudes y lógica del sistema
Servidores multimedia	Gestionan streaming de música y podcasts
Base de datos	Almacena información del sistema
Balanceadores de carga	Distribuyen tráfico entre servidores
CDN	Acelera entrega de contenido multimedia

Sistemas de respaldo	Protegen información ante fallos
----------------------	----------------------------------

2. Hosting y nube

Debido a la gran cantidad de usuarios y archivos multimedia, el sistema podría implementarse mediante servicios en la nube como Amazon Web Services, Google Cloud y Microsoft Azure.

3. Publicación

La publicación del sistema incluiría tanto la aplicación web como las aplicaciones móviles. En aplicación web se publicará mediante dominio oficial, certificación mediante HTTPS, servidores web y bases de datos configuradas.

En aplicación móvil se publicará en Google Play y App Store.

4. Capacitación de usuarios

Para facilitar el uso del sistema sería importante ofrecer capacitación y apoyo a los usuarios y administradores

Formas de capacitación:

1. Tutoriales interactivos.
2. Manuales digitales.
3. Videos explicativos.
4. Centro de ayuda.
5. Soporte técnico en línea.

Capacitación para administradores:

1. Gestión de usuarios,
2. Supervisión del sistema.
3. Control de contenido
4. Manejo de reportes y seguridad.
5. Migración de información

La migración de información consistiría en trasladar datos necesarios al nuevo sistema como por ejemplo:

1. Cuentas de usuario.
2. Playlists.
3. Historial de reproducción
4. Información de artistas.
5. Contenido multimedia.

Proceso de migración

1. Respaldo de información existente.
2. Validación de datos.
3. Conversión de formatos si es necesario.
4. Importancia a la nueva base de datos.
5. Verificación de datos de datos.

Mantenimiento.

1. Corrección de errores

Después de la implementación, pueden aparecer fallos que no fueron detectados durante las pruebas. Sin embargo tenemos soluciones para estos errores como solución de errores en reproducción de música, corrección de fallas en inicio de sesión, reparación de errores en sincronización de playlists, solución de problemas de compatibilidad entre dispositivos y corrección de fallos en notificaciones y búsquedas.

2. Actualizaciones

El sistema necesitaría tipos de actualizaciones constantes para mantenerlo moderno las cuales serian lo siguiente:

1. Actualización de interfaz gráfica.
2. Mejora de rendimiento.
3. Compatibilidad con nuevas versiones de Android y iOS.
4. Actualización de servidores y bases de datos.
5. Renovación de librerías y frameworks.

3. Nuevas funcionalidades

Con el tiempo se agregaran nuevas funciones para mejorar la plataforma y competir con otros servicios de streaming. Las posibles futuras actualizaciones serán las siguientes:

1. Podcasts de vídeo.
2. Modo karaoke.
3. Letras sincronizadas en tiempo real.
4. Recomendaciones mediante inteligencia artificial.
5. Chats o interacción social entre usuarios.

4. Seguridad

Las actividades de seguridad deben mantenerse constantemente actualizadas debido a las constantes amenazas.

1. Actualizar sistemas de autenticación.
2. Corrección de vulnerabilidades
3. Monitoreo de accesos sospechosos
4. Respaldo automático de información.
5. Renovación certificada de HTTPS.

5. Optimización

Para la optimización se busca mejorar el rendimiento y eficiencia del sistema donde las actividades consisten en reducir el tiempo de carga, optimizar la consulta de la base datos, mejorar la velocidad de streaming, reducción de consumo de recursos y balancear la carga entre servidores.

Análisis y reflexión

¿Qué etapa del SDLC consideras más importante y por qué?

Considero que la etapa más importante del SDLC es el análisis y planeación, porque en esta fase se definen los objetivos, requerimientos y necesidades del sistema SoundFlow. Si esta etapa se realiza incorrectamente, el proyecto podría presentar problemas desde el inicio, como funciones innecesarias, errores de diseño, aumento de costos o incumplimiento de necesidades del usuario.

¿Qué riesgos existirían si se omite una etapa?

Omitir una etapa del ciclo de vida del software puede provocar problemas graves durante el desarrollo y funcionamiento del sistema. Por ejemplo, si se omiten las pruebas, podrían liberarse errores importantes que afecten la experiencia del usuario o la seguridad de la información.

¿Por qué el mantenimiento es crítico en sistemas modernos?

El mantenimiento es crítico porque los sistemas modernos necesitan actualizarse constantemente para seguir funcionando correctamente y adaptarse a nuevas tecnologías y necesidades de los usuarios. Además, las amenazas cibernéticas cambian constantemente, por lo que un sistema sin mantenimiento puede volverse vulnerable y perder confiabilidad.

¿Cómo ayuda el SDLC a mejorar calidad y organización?

El SDLC ayuda a mejorar la calidad y organización porque divide el desarrollo del software en etapas estructuradas y controladas. También permite detectar problemas tempranamente, optimizar recursos y asegurar que el software cumpla con las necesidades del usuario y los estándares de calidad.